Proyecto 1 – Entrega 2: Documento de Diseño

Club Deportivo Séneca (CDS)

Integrantes:

- Jung Hann Chu Méndez 202412646

- John Edward Rincon Aragon 202420895

-David Alejandro Reyes Herrera 202521496

1. Introducción

Este documento presenta el diseño del sistema del Club Deportivo Séneca para la Entrega 2. Se basa en el diagrama de clases de diseño elaborado por el grupo e incluye dos diagramas de clases (alto nivel y diseño detallado), la descripción de los paquetes y clases, las decisiones de diseño, diagramas de secuencia de las funcionalidades críticas, la persistencia y el manejo de errores.

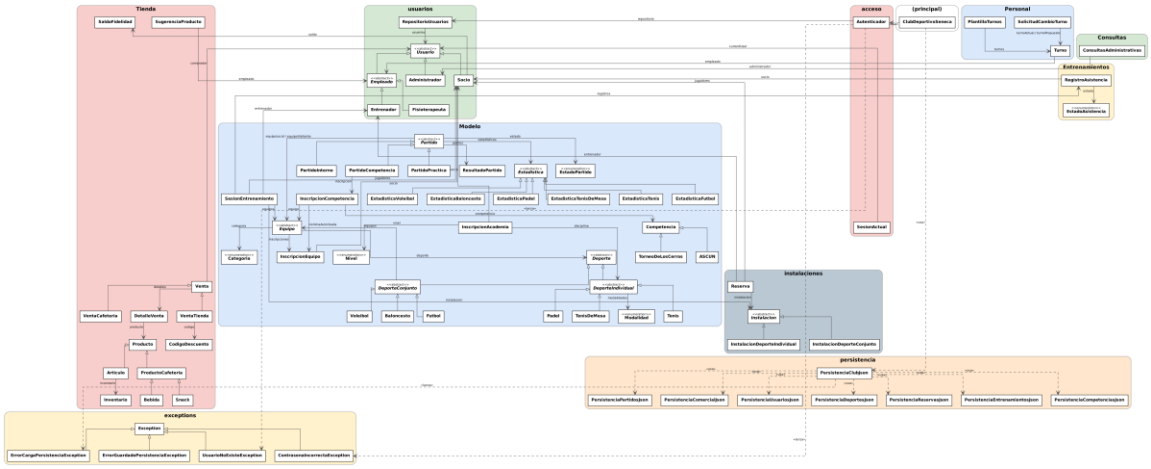
2. Arquitectura

El sistema está organizado en once paquetes. *ClubDeportivoSeneca* es el punto de entrada, este inicia la aplicación, autentica al usuario a través del paquete *acceso* y carga y guarda el estado con *PersistenciaClubJson*.

Paquete	Responsabilidad	*Clases*
(principal)	Punto de entrada de la aplicación	*ClubDeportivoSeneca*
usuarios	Usuarios del sistema y su registro	*Usuario, Socio, Empleado, Entrenador, Fisioterapeuta, Administrador, RepositorioUsuarios*
acceso	Inicio de sesión y usuario activo	*Autenticador, SesionActual*
Modelo	Deportes, equipos, inscripciones, partidos, competencias, sesiones y estadísticas	*Deporte y subclases, Equipo, InscripcionEquipo, InscripcionAcademia, Partido y subclases, ResultadoPartido, Competencia, ASCUN, TorneoDeLosCerros, InscripcionCompetencia, SesionEntrenamiento, Estadistica y subclases, enumeraciones*
Entrenamientos	Asistencia y evaluación en las sesiones	*RegistroAsistencia, EstadoAsistencia*
instalaciones	Canchas, mesas y reservas	*Instalacion, InstalacionDeporteIndividual, InstalacionDeporteConjunto, Reserva*

Paquete	Responsabilidad	*Clases*
Tienda	Productos, inventario, ventas, descuentos y fidelidad	*Producto, Articulo, ProductoCafeteria, Bebida, Snack, Venta, VentaTienda, VentaCafeteria, DetalleVenta, Inventario, CodigoDescuento, SugerenciaProducto, SaldoFidelidad*
Personal	Turnos de los empleados	*Turno, SolicitudCambioTurno, PlantillaTurnos*
Consultas	Consultas y reportes del administrador	*ConsultasAdministrativas*
persistencia	Carga y guardado en archivos JSON	*PersistenciaClubJson y siete clases de persistencia por tema*
exceptions	Errores de persistencia y de autenticación	*ErrorCargaPersistenciaException, ErrorGuardadoPersistenciaException, UsuarioNoExisteException, ContrasenaIncorrectaException*

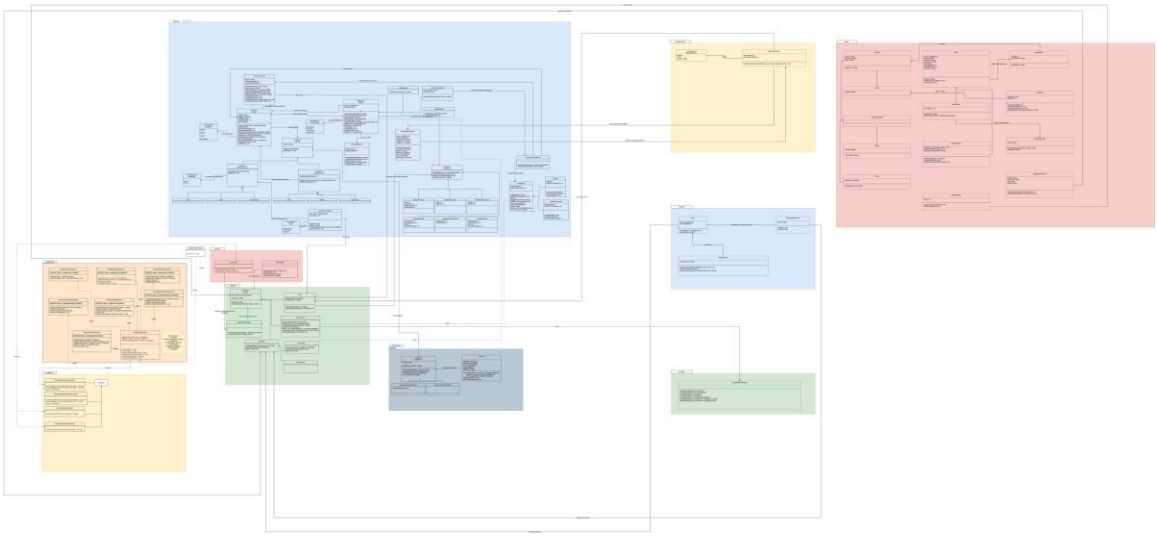
3. Diagrama de clases de alto nivel



Las relaciones principales son:

- *Usuario* es la raíz de los usuarios; *Empleado* se especializa en *Entrenador* y *Fisioterapeuta*.
- Un *Socio* se relaciona con los equipos a través de *InscripcionEquipo* y con los deportes individuales a través de *InscripcionAcademia*.
- *Partido* tiene tres tipos (interno, de competencia y de práctica) y guarda un *ResultadoPartido* y las *Estadisticas* de cada jugador.
- *Competencia* tiene dos formatos, *ASCUN* y *TorneoDeLosCerros*, y los equipos se inscriben mediante *InscripcionCompetencia*.
- *Venta* se divide en *VentaTienda* y *VentaCafeteria*, y se compone de objetos DetalleVenta.

4. Diagrama de clases de diseño



4.1 Métodos clave por clase

Clase	Método	Qué hace
InscripcionEquipo	inscribirSocio(socio, equipo, posicion, numeroCamiseta)	Inscribe a un socio en un equipo con su posición y número
InscripcionEquipo	consultarEquiposElegibles(socio, deporte)	Lista los equipos en los que el socio puede inscribirse
InscripcionEquipo	estaHabilitado(fecha), agregarSuspension(), cumplirSuspension()	Controla las suspensiones del jugador
Equipo	admiteEdad(fechaNacimiento)	Verifica que la edad corresponda a la categoría
Equipo	numeroCamisetaDisponible(numero)	Verifica que el número no esté en uso
Equipo	getJugadoresHabilitados(fecha), cumpleSuspensiones()	Calcula los jugadores que pueden jugar
InscripcionAcademia	inscribirSocio(socio), actualizarNivel()	Inscribe al socio en un deporte individual y maneja su nivel y ELO
Partido	esEnfrentamientoValido(), registrarResultado(...), iniciarPartido(), finalizarPartido()	Valida el partido y controla su estado

Clase	Método	Qué hace
PartidoPractica	confNivelesCompatibles(), actualizarRanking()	Aplica la regla de niveles y actualiza el ranking
PartidoCompetencia	validarParticipante(jugador)	Verifica que el jugador esté en la nómina
Competencia	getRequisitos(), admiteEquipo(equipo), inscribirEquipo(equipo, nomina)	Valida e inscribe equipos
ASCUN	generarEnfrentamientos(), avanzarRonda()	Maneja el formato por fases
TorneoDeLosCerros	actualizarTabla(), getTablaPosiciones()	Maneja el formato de liga
Estadistica	registrarEstadistica(jugador, estadistica), generaSuspension()	Registra estadísticas e indica suspensiones
RegistroAsistencia	registrar(estado, nota, comentario)	Registra asistencia y evaluación
Instalacion	admiteDeporte(deporte), getCapacidad(modalidad), estaDisponible(inicio, fin)	Valida la reserva de la instalación
Venta	liquidar(), redimirPuntos(cantidad), asignarPuntos()	Calcula el total y maneja los puntos
VentaTienda	calcularIVA(), aplicarCodigo(codigo)	Aplica IVA y descuento de empleado
VentaCafeteria	informarAlergenos(), calcularImpuesto(), ajustarPropina(porcentaje)	Aplica impuesto al consumo, propina y alerta de alérgenos
Inventario	descontar(cantidad), reabastecer(cantidad), trasladar(destino, cantidad)	Gestiona el stock
PlantillaTurnos	asignarTurno(empleado, turno), verificarCobertura(), aprobarCambio(solicitud)	Garantiza el mínimo de personal en turno
Autenticador	autenticar(email, contrasena)	Inicia sesión o lanza la excepción correspondiente
ConsultasAdministrativas	reporteVentas(inicio, fin), reporteDeportivo(inicio, fin)	Genera los reportes del administrador

5. Decisiones de diseño

En el apartado de diseño las decisiones a destacar fueron las siguientes:

- Omisión del registro independiente para los usuarios a causa de que el proyecto no explica si un usuario puede registrarse libremente, decidimos dejar que el socio solo pueda inscribir su fecha de cumpleaños, pero la única forma de registrar correo y contraseña para un socio es por medio de un administrador.

- Implementación de un repositorio central de usuarios debido a que diferentes partes del sistema necesitan consultar la misma información. Decidimos almacenar los usuarios en un HashMap utilizando su correo como identificador, evitando que cada clase tenga su propia lista de usuarios y facilitando la búsqueda durante la autenticación.

- Separación de responsabilidades entre el autenticador y los repositorios debido a que el autenticador únicamente debería encargarse de comprobar las credenciales de un usuario. Por esto decidimos que consulte los datos mediante los mecanismos existentes sin tener que cargar o administrar directamente todos los repositorios del club.

- Implementación de una clase SesionActual para almacenar el usuario autenticado, evitando que cada funcionalidad tenga que solicitar nuevamente su correo y contraseña. De esta forma, el sistema puede consultar el usuario que está utilizando el programa y comprobar sus permisos según el rol al que pertenece.

- Restricción de la modificación de la fecha de nacimiento debido a que esta información determina las categorías deportivas en las que puede participar un socio. Decidimos permitir que el socio la registre una sola vez y pueda consultarla posteriormente, dejando cualquier modificación a cargo del administrador para evitar cambios arbitrarios de categoría.

- Utilización de clases diferentes para los deportes de conjunto e individuales debido a que sus formas de participación son distintas. Mientras los deportes de conjunto requieren equipos, categorías y cupos, los individuales utilizan inscripciones por jugador, niveles y rankings, por lo que decidimos mantener estas responsabilidades separadas.

- Implementación de persistencia mediante archivos JSON debido a que el proyecto necesita conservar los datos sin depender de una base de datos. Decidimos separar la información por módulos y cargarla automáticamente al iniciar el programa, sin solicitar al usuario que seleccione carpetas o archivos.

- Separación de las carpetas /temp y /current debido a que un error durante el guardado podría dejar archivos incompletos o inconsistentes. Decidimos escribir primero toda la información en /temp y publicar el conjunto únicamente cuando esté completo, conservando el último estado válido si ocurre algún error.

- Separación de responsabilidades entre las ventas y el inventario debido a que registrar una compra y administrar las existencias son operaciones distintas. Decidimos mantener el control de cantidades y ubicaciones en el inventario, mientras las ventas se encargan de registrar los productos comprados, descuentos e impuestos correspondientes.

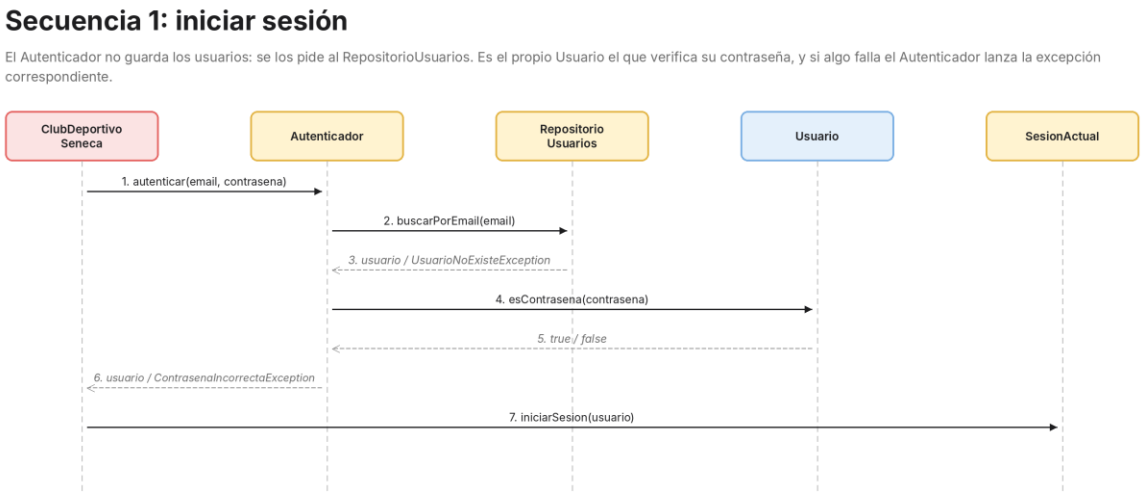
-Se decidió modelar las ubicaciones de inventario como una clase independiente, para que varios registros de inventario puedan referenciar un mismo objeto si se trata de una misma ubicación, y evitar así errores de tipeo o demás confusiones si la ubicación se guardara como un String.

-SaldoFidelidad se modelo como clase independiente, con composición a socio y empleado, pues administrador no tiene puntos de fidelidad. Así cada usuario de estos tipos tiene un objeto asociado que guarda la información de sus puntos y provee métodos útiles para calcularlos, sumarlos y redimirlos.

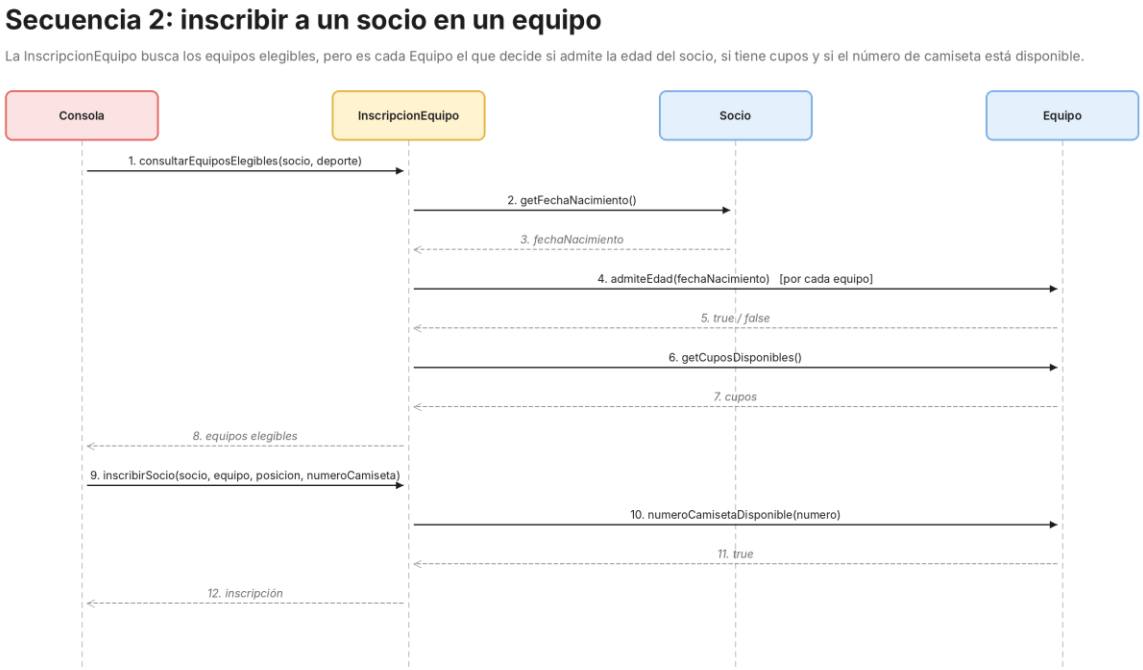
Al finalizar la implementación tuvimos que reformular el diseño de nuestro UML porque hubo varias cosas que no habíamos considerado. Adjuntamos los diagramas en la carpeta docs del repositorio.

6. Diagramas de secuencia.

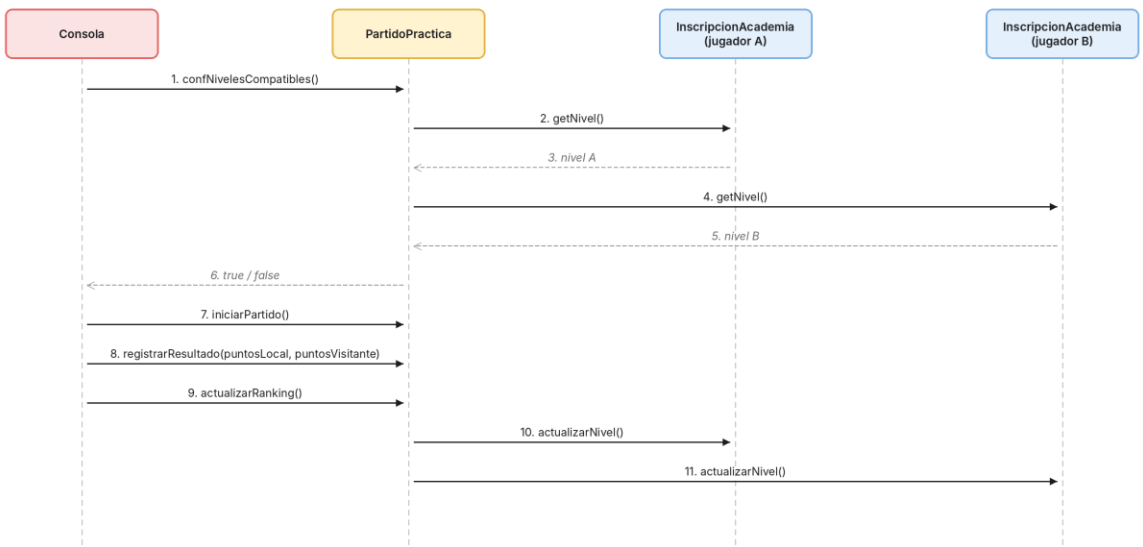
6.1 Inicio de sesión



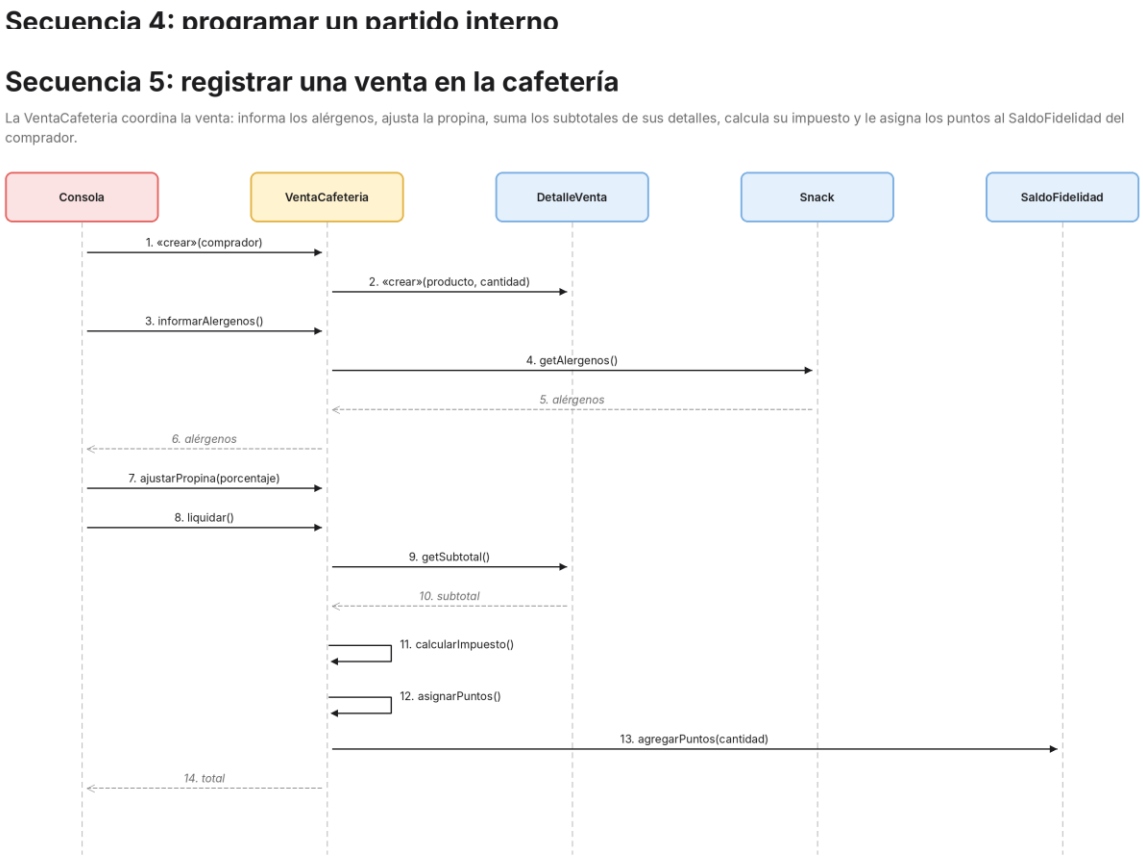
6.2 Socio se inscribe en un equipo (HU Socio 1)



6.3 Programar partido de práctica (HU Socio 6)



6.4 Programar partido interno (HU Administrador 1)



6.5 Venta en la cafetería

7. Persistencia

Los datos se guardan en archivos JSON dentro de un directorio de datos separado del código. *PersistenciaClubJson* coordina la carga y el guardado del club completo (cargarClub(), salvarClub()), valida la integridad de los datos y confirma el guardado. Cada tema tiene su propia clase y su propio archivo.

Orden de carga	*Clase*	Archivo	Contenido
1	*PersistenciaUsuariosJson*	usuarios.json	Socios, empleados y administrador
2	*PersistenciaDeportesJson*	deportes.json	Deportes, equipos e inscripciones
3	*PersistenciaReservasJson*	reservas.json	Instalaciones y reservas

Orden de carga	*Clase*	Archivo	Contenido
4	*PersistenciaCompetenciasJson*	competencias.json	Competencias
5	*PersistenciaPartidosJson*	partidos.json	Partidos y estadísticas
6	*PersistenciaEntrenamientosJson*	entrenamientos.json	Sesiones de entrenamiento
7	*PersistenciaComercialJson*	comercial.json	Productos, inventario y ventas

El orden importa: cada archivo referencia objetos cargados antes. Por ejemplo, los equipos necesitan a los socios ya cargados y los partidos necesitan a los equipos.

8. Manejo de errores

Todas las excepciones propias heredan de *Exception*, así que son verificadas: quien llama debe capturarlas o declararlas.

Excepción	Cuándo se lanza	Quién la lanza
ErrorCargaPersistenciaException	Falla la lectura de un archivo JSON o los datos no son válidos	Clases de *persistencia* al cargar
ErrorGuardadoPersistenciaException	Falla la escritura de un archivo	Clases de *persistencia* al guardar
UsuarioNoExisteException	El email no está registrado	*Autenticador*
ContrasenaIncorrectaException	La contraseña no coincide	*Autenticador*

Las reglas de negocio (cupos, categorías, niveles, mínimo de jugadores, disponibilidad de instalaciones) se validan con métodos que devuelven un booleano, como *admiteEdad*, *confNievelesEsCompatible*, *esEnfrentamientoValido* o *estaDisponible*.

9. Pruebas de las funcionalidades

Cada funcionalidad se probará cargando los datos con *PersistenciaClubJson*, ejecutando un caso exitoso y uno de rechazo, y mostrando el estado antes y después.

Funcionalidad	*Clases que se prueban*	Caso de rechazo
Inicio de sesión	*Autenticador, RepositorioUsuarios*	Email inexistente y contraseña incorrecta
Inscripción en equipo	*InscripcionEquipo, Equipo*	Edad fuera de categoría, sin cupo, número de camiseta en uso

Funcionalidad	*Clases que se prueban*	Caso de rechazo
Deportes individuales	*InscripcionAcademia, PartidoPractica*	Niveles no adyacentes sin entrenador
Reservas	*Instalacion, Reserva*	Instalación ocupada o modalidad no permitida
Entrenamientos	*SesionEntrenamiento, RegistroAsistencia*	Nota fuera de rango
Partidos y competencias	*PartidoInterno, Competencia, ASCUN, TorneoDeLosCerros*	Equipos de distinta categoría o sin mínimo de jugadores
Estadísticas	*Estadistica y subclases*	Suspensión por tarjetas
Ventas	*VentaTienda, VentaCafeteria, CodigoDescuento, SaldoFidelidad*	Bebida caliente en instalación no permitida
Turnos	*PlantillaTurnos, SolicitudCambioTurno*	Cambio que deja menos personal del mínimo
Consultas	*ConsultasAdministrativas*	No aplica
Persistencia	*PersistenciaClubJson*	Archivo dañado

9. Uso de IA generativa

La IA se usó principalmente para corregir errores en la construcción de la persistencia porque no estaba funcionando, el primer prompt usando el agente de codigo de openAI codex fue el siguiente:

“La persistencia no está funcionando corrige los errores y explica que hace falta”
resultado: Codex hizo ajustes en los archivos ya creados y dijo que sugiere agregar las siguientes clases; Json.java, PersistenciClub.java, ValidadorPersistencia.java.

Al pedir explicación se uso el siguiente prompt:
“Revisa los archivos que implementaste en Persistencia y explicame el por que de cada uno y porque la persistencia como la teniamos antes no funcionaba ”
Respuesta:

1. Json.java

Es una utilidad interna para leer y escribir el formato JSON usado por las demás clases de persistencia.

Sus responsabilidades son:

- Leer un archivo con un arreglo JSON.
- Convertir cada objeto en un Map<String, String>.
- Convertir mapas nuevamente en objetos JSON.

- Escapar comillas, barras y saltos de línea.
- Detectar estructuras básicas inválidas, como objetos o cadenas sin cerrar.

Por ejemplo, convierte esto:

```
[
 {
  "tipo": "SOCIO",
  "email": "socio@club.com"
 }
]
```

en una estructura equivalente a:

List<Map<String, String>>

¿Por qué existe?¿Por qué existe?

Porque todas las clases como PersistenciaUsuariosJson, PersistenciaDeportivoJson y PersistenciaPartidosJsonnecesitan hacer las mismas operaciones JSON. Sin esta clase, cada una tendría que repetir la lectura, escritura, escape y análisis del archivo.

Además, el proyecto no tiene una biblioteca JSON como Jackson o Gson. Esta clase evita agregar una dependencia externa.

Limitación importanteLimitación importante

No es un lector JSON general. Solamente soporta el formato controlado del proyecto:

- Un arreglo en la raíz.
- Objetos planos.
- Valores guardados como cadenas.
- Sin objetos o arreglos anidados.

Eso funciona porque las relaciones se guardan mediante identificadores, por ejemplo:

```
{
 "socio": "correo@club.com",
 "equipo": "Equipo A"
}
```

No se intenta guardar un objeto Socio completo dentro del equipo.

2. ValidadorPersistencia.java

Comprueba que los siete archivos constituyan un estado completo y coherente antes de modificar los registros en memoria.

Valida, entre otras cosas:

- Usuarios duplicados.
- Tipos de usuario válidos.
- Fechas y suspensiones válidas.
- Equipos asociados con deportes existentes.
- Inscripciones asociadas con socios y equipos existentes.
- Instalaciones y reservas coherentes.
- Competencias asociadas con deportes y equipos existentes.
- Partidos con entrenador, deporte y participantes válidos.
- Estadísticas asociadas con un partido y socio existentes.
- Sesiones asociadas con entrenador, instalación y deporte.
- Calificaciones entre 1 y 5.
- Valores válidos de enums como Categoria, Modalidad y FormatoCompetencia.

¿Por qué debe validar antes de cargar?¿Por qué debe validar antes de cargar?

Muchas clases del modelo se registran automáticamente dentro de colecciones estáticas al ejecutar sus constructores. Por ejemplo, construir un Socio, Equipo o Deporte ya modifica el estado global.

Sin validación previa podría ocurrir esto:

1. Se cargan correctamente diez usuarios.
2. Se cargan dos deportes.
3. Se encuentra una inscripción que referencia un equipo inexistente.
4. La carga falla.
5. Los usuarios y deportes anteriores ya quedaron registrados.

El resultado sería un club parcialmente cargado.

ValidadorPersistencia evita eso comprobando primero todos los archivos y sus relaciones. Solo después de que todo sea válido, PersistenciaClubJson limpia el estado anterior y empieza a reconstruirlo.

Por qué usa mapas temporalesPor qué usa mapas temporales

Los mapas de usuarios, deportes, equipos, competencias y sesiones no son el estado real del club. Son índices temporales para comprobar referencias.

Por ejemplo:

if (!equipos.containsKey(requerido(r, "equipo")))

permite detectar que una inscripción apunta a un equipo inexistente antes de construir la inscripción.

3. PersistenciaClubJson.java

Es el coordinador principal de toda la persistencia.

No conoce en detalle cómo se serializa cada tipo de dato. Esa responsabilidad está distribuida entre:

- PersistenciaUsuariosJson
- PersistenciaDeportivoJson

- PersistenciaReservasJson
- PersistenciaCompetenciasJson
- PersistenciaPartidosJson
- PersistenciaEntrenamientosJson
- PersistenciaComercialJson

Su responsabilidad es coordinar el proceso completo.

Al cargarAl cargar

Realiza este proceso:

1. Crea las carpetas current y temp.
2. En la primera ejecución crea los siete JSON vacíos.
3. Comprueba que existan todos los archivos obligatorios.
4. Lee todos los archivos.
5. Ejecuta ValidadorPersistencia.
6. Si todo es válido, limpia las colecciones estáticas.
7. Reconstruye los objetos en el orden adecuado.

El orden importa:

1. Usuarios.
2. Deportes y equipos.
3. Instalaciones y reservas.
4. Competencias.
5. Partidos y estadísticas.
6. Entrenamientos.
7. Comercial.

Por ejemplo, una inscripción no se puede reconstruir antes de que existan su socio, deporte y equipo.

Al guardarAl guardar

No escribe directamente sobre current. Primero guarda todo dentro de temp:

```
persistencia/
 ├── current/
 └── temp/
```

Después valida que temp tenga todos los archivos y que sean JSON legibles. Finalmente:

1. Mueve current a current-anterior.
2. Mueve temp a current.
3. Si falla el segundo movimiento, recupera current-anterior.
4. Si todo sale bien, elimina la copia anterior.

Esto hace que el guardado sea prácticamente atómico: o queda publicado el conjunto nuevo completo, o continúa disponible el anterior.

ReintentosReintentos

salvarClub() programa otro intento si falla el guardado.

salvarClubHastaConfirmar() continúa intentando cada dos minutos hasta guardar correctamente. Este último se usa al cerrar el programa para cumplir la regla de no terminar sin intentar conservar la información.

Una consecuencia deliberada es que el cierre puede quedar esperando indefinidamente si el disco nunca vuelve a estar disponible.

Por qué la persistencia anterior no funcionabaPor qué la persistencia anterior no funcionaba

El historial de Git muestra algo muy concreto: antes del commit de persistencia no existía una implementación real.

La prueba anterior decía:

@Disabled("No existe una implementación de persistencia")

y sus tres pruebas solo contenían comentarios pendientes:

- Guardar y recargar el estado.
- Evitar una carga parcial.
- Mantener el estado anterior cuando falla el guardado.

Por tanto, anteriormente:

- Los datos solamente vivían en las colecciones estáticas de Java.
- Al cerrar el programa se perdían.
- No existían clases que reconstruyeran las relaciones.
- No había archivos JSON reales.
- No se validaban referencias entre archivos.
- No había protección frente a guardados incompletos.
- Las pruebas estaban deshabilitadas porque no había nada ejecutable que probar.

La estructura que habías diseñado en el UML sí separaba correctamente los dominios, pero era una estructura conceptual. Faltaban tres piezas esenciales que ahora aportan estos archivos:

Json
↓ convierte texto ↔ registros

ValidadorPersistencia
↓ verifica todo antes de modificar memoria

PersistenciaClubJson
↓ coordina orden, carga y publicación atómica

Persistencias específicas
↓ reconstruyen usuarios, deportes, partidos, etc.

Si simplemente se hubiera llamado consecutivamente a cada persistencia específica, también habría existido un problema: si el quinto archivo fallaba, los cuatro primeros ya habrían modificado el club. Por eso ValidadorPersistencia y la estrategia current/temp no son decorativas; solucionan los dos fallos principales:

- Estado parcial durante la carga.
- Archivos parcialmente reemplazados durante el guardado.

También en su implementación vimos que hay métodos donde los atributos no tienen visibilidad en el archivo, buscando en google gemini respondió:
Pregunta: que significa que un método en java no tenga private ni public
Resultado: Que un método en Java no tenga private ni public (y tampoco protected) significa que tiene un modificador de acceso por defecto (conocido en inglés como *default* o *package-private*).

Apartado de tienda, la IA en el apartado de tienda tuvo varias falencias, en lugar de mejorar el código, sentimos que hizo cosas sin sentido, en este caso, después de la implementación del package de pruebas, el agente sugirió mejorar funciones en tienda agregando un archivo de UbicacionInventario nuevo, pero dejando el anterior archivo como @Deprecated esto nos parece particularmente curioso porque su código si resuelve las pruebas satisfactoriamente pero dejamos de confiar en la calidad del agente para usarlo si su tarea será "mejorar" nuestras implementaciones.
Prompt: "Arregla la tienda donde pueda haber errores difíciles de evidenciar y mejora el código actual en caso de que sea necesario."

Uso adicional: después de terminar cada implementación le pedimos al agente que mejorará el código y lo hiciera más fácil de entender y escalar.
Prompt: "Revisa la implementación actual mencionandonos los errores críticos del mismo y como lo solucionaste"
Las respuestas que más aprendizaje nos dejaron:

1. El caso más critico : Se cargaban los archivos directamente en este orden:
   usuarios → deportes → equipos → reservas → partidos

   y aparecía un error al cargar partidos, todo lo anterior ya habría sido insertado en las colecciones estáticas.

   El club quedaba mezclando:

- Datos antiguos.
- Datos nuevos cargados parcialmente.
- Objetos duplicados.
- Referencias que podían no coincidir.

2. Sobrescribir directamente archivos como usuarios.json o deportivo.json era peligroso.Si el programa fallaba después de escribir usuarios, pero antes de escribir partidos, quedaba una versión mezclada.

3. No se validaron las relaciones antes de construir los objetos.

4. Recorridos adicionales en varios fragmentos del código.

Por último el agente nos ayudó a usar la maqueta de pruebas que le dimos para construir pruebas más sólidas con base en nuestros requerimientos funcionales.
Prompt: “Usa esta maqueta de pruebas para mejorar las pruebas e anadir los casos base ignorados”

Explicación de nuestros errores: Las pruebas compartían colecciones estáticas entre ejecuciones, no siempre preparaban correctamente las relaciones entre socio, deporte y equipo, y varios rechazos solo comprobaban la excepción, pero no que el estado permaneciera intacto. Las adapté a la arquitectura actual sin ampliar innecesariamente su alcance: actualicé las llamadas, preparé escenarios mínimos válidos, limpié los repositorios en @BeforeEach y comprobé tanto los resultados exitosos como la ausencia de cambios después de operaciones rechazadas.

También se utilizó IA generativa para crear el esqueleto del doumento actual.